M A T E R I A

Base de Datos NoSQL

Documentación técnica

Migración de SQL a MongoDB Atlas y aseguramiento de la base de datos

Proyecto: Salón de Eventos “Los Chaneques”

I N T E G R A N T E S

Marco Aurelio Somarriba Cadena

Ángel de Jesús López Campechano

19 de octubre de 2026

https://www.rentas.me

Índice

1. Introducción y alcance
2. Arquitectura y herramientas
3. Diagrama de la base de datos relacional (origen)
4. Diagrama del modelo NoSQL (MongoDB)
5. Punto 1: Migración de SQL a MongoDB
6. Punto 5: Consultas CRUD
7. Punto 2: Seguridad, control de acceso y autenticación
8. Punto 3: Cifrado en tránsito de datos
9. Punto 4: Hardening de red (VPN, firewall, SSL y TLS)
10. Punto 7: Servicios del GitHub Student Pack
11. Punto 6: Comandos utilizados
12. Conclusiones y pendientes

1. Introducción y alcance

Este documento explica cómo se realizó el proyecto de la materia de **Base de Datos NoSQL**: la base de datos relacional del sitio web del salón de eventos **Los Chaneques** (PostgreSQL) se respaldó, se convirtió a un modelo de documentos y se puso en producción sobre **MongoDB Atlas**, con medidas de seguridad y publicación en internet con dominio y certificado propios.

El jefe de proyecto definió siete puntos que la base de datos debe cumplir. La siguiente tabla indica dónde se atiende cada uno:

#	Requisito del jefe de proyecto	Sección	Estado
1	Migración de SQL a MongoDB	3, 4 y 5	✅ Cumplido
2	Seguridad: control de acceso y autenticación	7	✅ Cumplido (usuarios de BD por crear)
3	Cifrado en tránsito de datos	8	✅ Cumplido
4	Hardening de red (VPN, firewall, SSL, TLS)	9	🟡 VPN pendiente
5	Consultas CRUD	6	✅ Cumplido
6	Comandos	11	✅ Cumplido
7	Al menos 2 servicios del GitHub Student Pack	10	✅ MongoDB Atlas y Namecheap

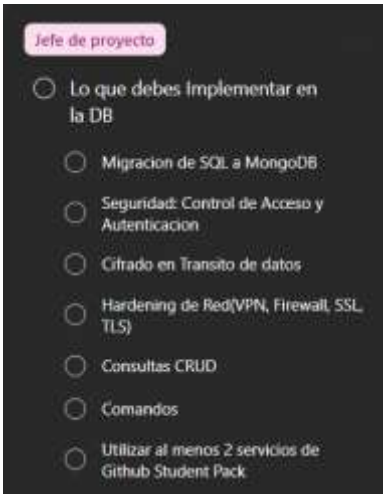


Figura 1. Lista de requisitos del jefe de proyecto.

2. Arquitectura y herramientas

El visitante entra por el dominio **rentas.me** al frontend (Vercel). El frontend consume una API propia (Render) que lee y escribe en **MongoDB Atlas**. Las fotos subidas desde el panel se guardan en Supabase Storage y la base PostgreSQL de Supabase fue el origen de la migración.

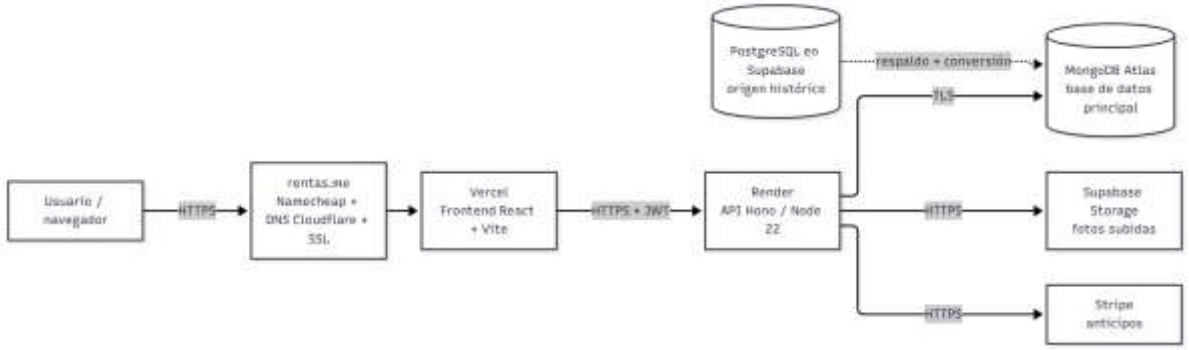


Figura 2. Arquitectura de la solución.

Capa	Tecnología / servicio
Base de datos	**MongoDB Atlas** (cluster `MiClusterBDNOSQL`, base `los_chaneques`)
Backend	Node 22, Hono, TypeScript; controlador oficial `mongodb`; desplegado en Render
Frontend	React 19, Vite, Tailwind; desplegado en Vercel
Dominio, DNS y SSL	**Namecheap** (dominio `rentas.me`), Cloudflare (DNS), Let's Encrypt (certificado)
Origen de la migración	PostgreSQL en Supabase
Código	GitHub (`los-chaneques` y `Migracion_SalonEventosMongoDB`)

3. Diagrama de la base de datos relacional (origen)

El sistema se diseñó primero como base relacional en PostgreSQL: 12 tablas con llaves primarias (PK), foráneas (FK) y únicas (UK). Las relaciones principales son: un usuario realiza muchas reservaciones, un paquete de

comida puede estar en muchas reservaciones (de forma opcional) y una conversación de chat contiene muchos mensajes.

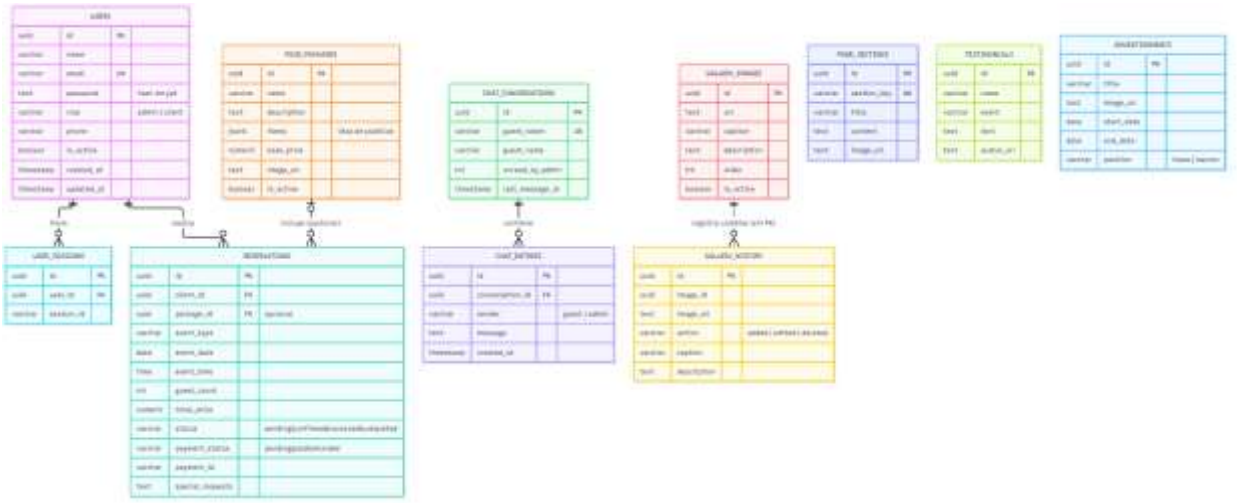


Figura 3. Diagrama entidad-relación (PostgreSQL).

Tabla	Contenido	Filas
users	Cuentas del administrador y de los clientes (contraseña con hash bcrypt)	6
food_packages	Paquetes de comida; los platillos se guardan como `jsonb`	7
reservations	Reservaciones; FK a `users` y `food_packages`	7
chat_conversations / chat_entries	Conversaciones y sus mensajes (1 a muchos)	10 / 94
gallery_images / gallery_history	Fotos con título y descripción, y su histórico de cambios	31 / 39
page_sections, testimonials, advertisements, user_sessions	Contenido del sitio, opiniones, anuncios y sesiones	3 · 4 · 0 · 0

Conteos del respaldo del 14 de octubre de 2026: 201 registros en total.

4. Diagrama del modelo NoSQL (MongoDB)

Al convertir a documentos se aplicaron dos patrones: **incrustar** los datos que siempre se leen juntos (mensajes dentro de la conversación) y **referenciar** los demás con el identificador (`clientId`, `packageId`). Las reservaciones además guardan una copia del nombre del cliente y del paquete (patrón de referencia extendida) para mostrar la lista sin cruzar colecciones.

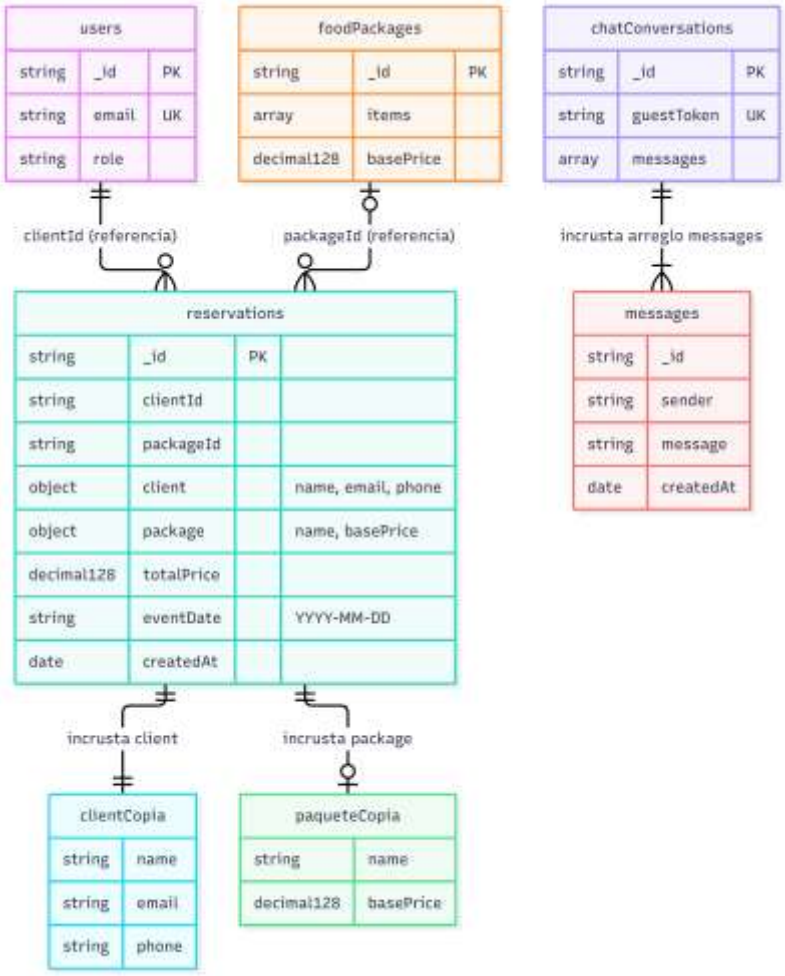


Figura 4. Modelo de documentos de MongoDB Atlas.

Tabla SQL	Colección MongoDB	Cambio principal
users	`users`	`_id` = UUID original
food_packages	`foodPackages`	Platillos como arreglo; precio en Decimal128
reservations	`reservations`	Incrusta `client` y `package`; conserva `clientId` y `packageId`
chat_conversations + chat_entries	`chatConversations`	Mensajes como arreglo `messages` dentro de la conversación
Las demás	mismo nombre en camelCase	Relación 1 a 1

Tipos: precios como `Decimal128` (sin errores de redondeo), fechas de registro como `Date` en UTC, y la fecha y hora del evento como texto `YYYY-MM-DD` / `HH:MM:SS` para no depender de la zona horaria. **Índices únicos** en `users.email`, `chatConversations.guestToken` y `pageSections.sectionKey` (reemplazan los `UNIQUE` de SQL) y **validadores** `$jsonSchema` en `users`, `foodPackages`, `reservations` y `chatConversations` (reemplazan los `NOT NULL`).

5. Punto 1: Migración de SQL a MongoDB

La migración se hizo en cinco pasos, siempre sobre una **copia** para no arriesgar la base original.

5.1 Respaldo de solo lectura

```
$PGD = "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe"
& $PGD "<URL_POSTGRES>" --schema=public --schema-only --no-owner -f
respaldo/estructura_los_chaneques.sql
& $PGD "<URL_POSTGRES>" --schema=public --no-owner --inserts -f
respaldo/respaldo_completo_los_chaneques.sql

PS> (Select-String "CREATE TABLE" respaldo\estructura_los_chaneques.sql).Count    →  12
PS> (Select-String "^INSERT INTO" respaldo\respaldo_completo_los_chaneques.sql).Count  →  201
```

5.2 Copia aparte y conversión a documentos

Con el script `scripts/convertir.mjs` se lee la base (solo lectura) y se generan los documentos en la carpeta `atlas_import/`, que es la copia aparte que se importa.

```
cd Migracion_Rentas
npm install pg mongodb bson dotenv
npm run convertir

Colecciones generadas: { users: 6, foodPackages: 7, reservations: 7, chatConversations: 12,
  pageSections: 3, galleryImages: 31, galleryHistory: 39, testimonials: 4, advertisements: 0,
userSessions: 0 }
```

5.3 Importación a MongoDB Atlas y verificación

El script `importar-atlas.mjs` crea las colecciones con sus validadores, inserta o reemplaza por `_id` (se puede repetir sin duplicar), crea los índices y compara conteos con el origen:

```
npm run importar

| coleccion           | origen | enAtlas | ok   |       | galleryImages | 31 | 31 | true |
| chatConversations   | 12     | 12      | true |       | reservations  | 7  | 7  | true |
| foodPackages        | 7      | 7       | true |       | testimonials  | 4  | 4  | true |
| users               | 6      | 6       | true |
✅ Importación completa y verificada.
```

5.4 La aplicación pasa a usar MongoDB

Después de los datos se migró el código: el backend se reescribió con el controlador oficial de MongoDB (antes usaba Drizzle ORM sobre PostgreSQL) y la variable de conexión cambió de `DATABASE_URL` a `MONGODB_URI`. La API devuelve exactamente la misma forma de datos, por lo que el frontend no cambió. Se probaron contra Atlas el inicio de sesión, las reservaciones, el chat, la galería con su histórico, los anuncios y la subida de imágenes, y los conteos finales coincidieron con los originales.

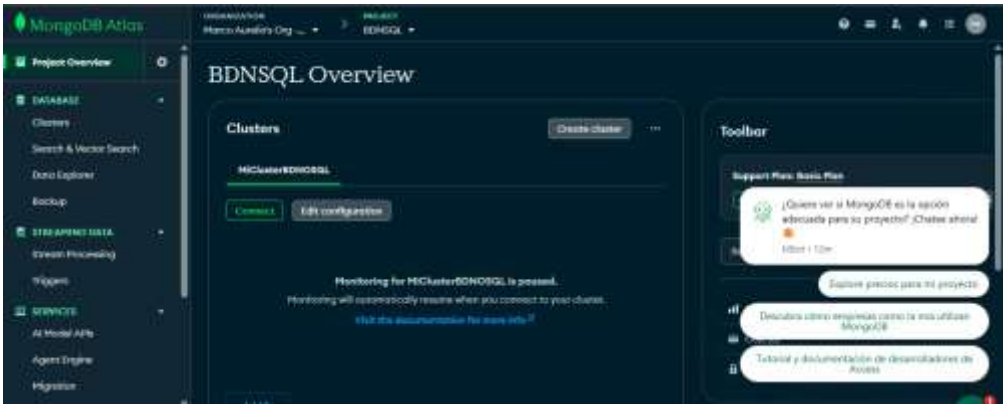


Figura 5. Proyecto BDNSQL y cluster MiClusterBDNOSQL en MongoDB Atlas.

6. Punto 5: Consultas CRUD

Las operaciones CRUD se ejecutaron contra el cluster de Atlas sobre una colección temporal (`demo_crud`) que se eliminó al terminar, y las lecturas sobre los datos reales. Se muestran en sintaxis de `mongosh`; los resultados son los obtenidos.

6.1 Create: insertar documentos

```
db.demo_crud.insertOne({ _id: "demo-1", nombre: "Paquete de prueba", precio: Decimal128("9999.00"),
                         items: ["Entrada", "Plato fuerte"], activo: true })
→ { acknowledged: true, insertedId: "demo-1" }

db.demo_crud.insertMany([ { _id: "demo-2", nombre: "Taquiza demo", precio: Decimal128("4500.00"),
items: ["Tacos"], activo: true },
                          { _id: "demo-3", nombre: "Postres demo", precio: Decimal128("2500.00"),
items: ["Pastel"], activo: false } ])
→ { acknowledged: true, insertedCount: 2 }
```

6.2 Read: consultar y agregar

```
db.demo_crud.find({ activo: true }, { nombre: 1, precio: 1 })
→ [ { _id: "demo-1", nombre: "Paquete de prueba", precio: 9999.00 }, { _id: "demo-2", nombre:
"Taquiza demo", precio: 4500.00 } ]

db.demo_crud.find({ precio: { $gt: Decimal128("3000") } }, { _id: 0, nombre: 1 })
→ [ { nombre: "Paquete de prueba" }, { nombre: "Taquiza demo" } ]

db.demo_crud.aggregate([ { $group: { _id: "$activo", total: { $sum: 1 }, suma: { $sum: "$precio" } }
} ])
→ [ { _id: true, total: 2, suma: 14499.00 }, { _id: false, total: 1, suma: 2500.00 } ]

// Sobre los datos reales
db.foodPackages.find({ isActive: true }, { _id: 0, name: 1, basePrice: 1 }).sort({ basePrice: -1
}).limit(3)
→ Menú Gourmet a 3 Tiempos 38000.00 · Buffet Mexicano Tradicional 24000.00 · Parrillada Ranchera
18500.00
db.reservations.aggregate([ { $group: { _id: "$status", total: { $sum: 1 } } } ])
→ [ { _id: "confirmed", total: 4 }, { _id: "pending", total: 3 } ]
```

6.3 Update: actualizar

```
db.demo_crud.updateOne({ _id: "demo-1" }, { $set: { precio: Decimal128("10500.00") }, $push: {
items: "Postre" } })
→ { matchedCount: 1, modifiedCount: 1 }
db.demo_crud.updateMany({ activo: false }, { $set: { activo: true } })
→ { matchedCount: 1, modifiedCount: 1 }
db.demo_crud.findOne({ _id: "demo-1" })
→ { _id: "demo-1", nombre: "Paquete de prueba", precio: 10500.00, items: ["Entrada", "Plato fuerte",
"Postre"], activo: true }
```

6.4 Delete: eliminar

```
db.demo_crud.deleteOne({ _id: "demo-2" })   → { deletedCount: 1 }
db.demo_crud.deleteMany({})                  → { deletedCount: 2 }
db.demo_crud.drop()                          → true
```

6.5 CRUD desde la aplicación

La API traduce cada petición HTTP a una operación de MongoDB, protegida por rol:

Operación	Ruta de la API	Método de MongoDB	Quién
Create	`POST /api/reservations/mine`	`insertOne`	Cliente con cuenta
Read	`GET /api/packages`	`find`	Público
Read	`GET /api/reservations`	`aggregate` con `$lookup`	Administrador
Update	`PUT /api/reservations/:id`	`findOneAndUpdate`	Administrador
Update	`POST /api/chat/guest/:token`	`updateOne` con `$push` y `$inc`	Visitante
Delete	`DELETE /api/gallery/:id`	`findOneAndDelete` (+ histórico)	Administrador

7. Punto 2: Seguridad, control de acceso y autenticación

Rol	Permisos
Visitante	Ver el sitio y usar el chat con un código anónimo guardado en su navegador
`client`	Crear su cuenta, reservar, ver y cancelar **solo sus** reservaciones y pagar su anticipo
`admin`	Panel completo: reservaciones, paquetes, galería, testimonios, anuncios, chat y editor del sitio

- **Autenticación:** token JWT firmado con un secreto del servidor (obligatorio en producción), con vigencia de 7 días.
- **Contraseñas:** hash `bcrypt` con 10 rondas; mínimo 8 caracteres; nunca en texto plano.
- **Autorización en el servidor:** los *middlewares* `requireUser` y `requireAdmin` verifican el rol firmado en el token en cada ruta.
- **Sin escalada de privilegios:** el registro siempre crea rol `client`; el rol `admin` solo se asigna con `npm run seed:admin`.
- **Anti fuerza bruta:** límite de intentos en inicio de sesión (15 cada 15 min) y registro (10 cada 15 min).

- **Validación de entradas:** lista blanca de campos al actualizar, imágenes limitadas a 5 MB y a tipos seguros, enlaces de anuncios restringidos.

Prueba	Resultado
`GET /api/admin/stats` sin sesión	**401** No autorizado
`GET /api/admin/stats` con sesión de cliente	**403** Acceso denegado
Reservar sin cuenta	**401** "Inicia sesión para continuar"
Cancelar la reservación de otro usuario	**404** "Reservación no encontrada"
Registro con contraseña de 3 caracteres	**400** "…al menos 8 caracteres"

Usuarios y roles dentro de MongoDB Atlas (mínimo privilegio)

Usuario de BD	Rol en Atlas	Uso
`app_backend`	`readWrite` sobre `los_chaneques`	Solo el backend
`analista`	`read` sobre `los_chaneques`	Consultas y reportes, sin modificar

Creación con Atlas CLI (o en Atlas → Database Access)

```
atlas dbusers create --username app_backend --password <PASSWORD> --role readWrite@los_chaneques
atlas dbusers create --username analista    --password <PASSWORD> --role read@los_chaneques
```

8. Punto 3: Cifrado en tránsito de datos

Todo el tráfico entre componentes viaja cifrado con **TLS**. Se comprobó abriendo una conexión TLS real a cada punto:

Tramo	Protocolo y cifrado (medido)	Certificado
Navegador → `www.rentas.me` (Vercel)	TLS 1.3 · TLS_AES_256_GCM_SHA384	Let's Encrypt, válido hasta el 17 ene 2027
Navegador → API `los-chaneques.onrender.com`	TLS 1.3 · TLS_AES_256_GCM_SHA384	Google Trust Services, válido hasta el 20 dic 2026
Backend → MongoDB Atlas (puerto 27017)	TLS 1.3 · TLS_AES_128_GCM_SHA256	Let's Encrypt, válido hasta el 24 dic 2026
Backend → Stripe / Supabase Storage	HTTPS	Certificados del proveedor

Evidencia

```
PS> openssl s_client -connect www.rentas.me:443 -servername www.rentas.me | openssl x509 -noout -
issuer -subject -dates
issuer=C=US, O=Let's Encrypt, CN=YE1
subject=CN=rentas.me
notBefore=Oct 19 17:01:38 2026 GMT
notAfter=Jan 17 17:01:37 2027 GMT

PS> curl.exe -I https://www.rentas.me
strict-transport-security: max-age=63072000; includeSubDomains; preload
```

La cadena de conexión de Atlas usa el formato `mongodb+srv://`, que **siempre activa TLS**; no existe una conexión sin cifrar. **HSTS** obliga al navegador a usar siempre HTTPS con el sitio.

Cifrado en reposo y de las contraseñas

- **MongoDB Atlas** cifra el almacenamiento en reposo (AES-256) por defecto; las contraseñas de los usuarios se guardan además como hash `bcrypt`, irreversible.
- Las claves y cadenas de conexión viven solo en variables de entorno (Render) o en archivos `.env` que no se suben a GitHub.

9. Punto 4: Hardening de red (VPN, firewall, SSL y TLS)

9.1 Firewall

Control	Implementación
Lista de IP de Atlas (*Network Access*)	Solo se permiten el equipo de desarrollo y las IP de salida de Render; sin `0.0.0.0/0`
CORS con lista blanca	Solo `localhost` (desarrollo) y los dominios de `FRONTEND_URL`; un origen externo no recibe `Access-Control-Allow-Origin`
Límite de tamaño	6 MB por petición; una subida de 7 MB devuelve **413**
Límite de peticiones	240 por minuto por ruta e IP; 429 al excederlo
Cabeceras de seguridad	`Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`
Política de contenido (CSP)	Solo scripts del propio sitio, imágenes de orígenes permitidos y conexiones a la API

Comprobación

```
PS> curl.exe -i http://localhost:3000/api/gallery
strict-transport-security: max-age=15552000; includeSubDomains
x-content-type-options: nosniff      x-frame-options: SAMEORIGIN
PS> curl.exe -i -H "Origin: https://evil.com" http://localhost:3000/api/gallery
(sin Access-Control-Allow-Origin → el navegador bloquea al origen externo)
```

9.2 SSL y TLS

El dominio **rentas.me** se sirve únicamente por HTTPS con certificado válido de Let's Encrypt (que cubre `rentas.me` y `www.rentas.me`), con TLS 1.3 y HSTS con *preload* (sección 8). El dominio sin "www" redirige con código 308 a `https://www.rentas.me/`. La API en Render y la conexión a Atlas también usan TLS 1.3.

10. Punto 7: Servicios del GitHub Student Pack

Servicio	Beneficio del Student Pack	Uso en el proyecto
MongoDB Atlas	Cluster y créditos para estudiantes	Base de datos principal: colecciones, índices, validadores, usuarios y firewall
Namecheap	Dominio `.me` gratuito por un año	Registro de **rentas.me**; el sitio se publica con HTTPS y certificado válido

Además se usaron servicios de capa gratuita: GitHub (repositorios), Vercel (frontend), Render (backend), Cloudflare (DNS) y Supabase (Storage de fotos y base de origen).

Configuración del dominio

- Dominio `rentas.me` registrado en Namecheap; servidores DNS de Cloudflare (`amy` y `amos.ns.cloudflare.com`).
- Frontend en Vercel con los registros DNS indicados por Vercel; Let's Encrypt emite el certificado automáticamente.
- `rentas.me` redirige a `https://www.rentas.me`, y la API se publica en `https://los-chaneques.onrender.com`.

11. Punto 6: Comandos utilizados

Fase	Comando
Verificar herramientas	`node -v · npm -v · git --version · pg_dump --version · atlas --version`
Respaldo	`pg_dump <URL> --schema=public --schema-only --no-owner -f estructura.sql` y `… --inserts -f respaldo_completo.sql`
Conversión	`npm run convertir`
Importación	`npm run importar`
Conexión a Atlas	`mongosh "mongodb+srv://<cluster>.mongodb.net/los_chaneques" --username <usuario>`
Usuarios y red de Atlas	`atlas dbusers create … · atlas accessLists create --currentIp`
Backend	`npm install mongodb · npm run dev · npm start · npm run seed:admin`
Frontend	`npm run dev · npm run build`
Comprobación del despliegue	`npm run comprobar -- https://los-chaneques.onrender.com https://www.rentas.me`
SSL y red	`openssl s_client -connect www.rentas.me:443 · curl.exe -I https://www.rentas.me · Resolve-DnsName rentas.me`
VPN	`winget install Tailscale.Tailscale · tailscale up --advertise-exit-node`
Código	`git init -b main · git add -A · git commit -m "…" · git push -u origin main`

El comando `npm run comprobar` pide a la API crear una conversación de prueba y la busca en Atlas y en PostgreSQL, lo que demuestra con certeza en qué base escribe el servicio publicado (resultado: **MongoDB Atlas**).

12. Conclusiones y pendientes

La base de datos relacional del salón de eventos se respaldó, se convirtió a un modelo de documentos y se puso en producción sobre MongoDB Atlas sin cambiar el comportamiento del sitio. El modelo NoSQL simplificó consultas que antes requerían uniones (reservaciones con su cliente y paquete, chats con sus mensajes) y la seguridad se resolvió en capas: roles y autenticación en la aplicación, TLS en todos los tramos, firewall y límites en la red.